接口 0 的 3/1/1 命中内置 usbkbd，同步 probe 并占用 ch6；接口 1 的 3/1/2 没有任何已注册驱动匹配（内置键盘只认 protocol=1），于是设备 core 走未匹配分支：用 bus 的 modalias() 生成别名并 request_module()。

表 7.6: Boot Mouse 的 MODALIAS 与 HID control、报告报文

项	内容
MODALIAS	usb:c03s01p02（class/subclass/protocol 的十六进制）；静态 alias 表映射到 usbmouse.ko
SET_PROTOCOL	SETUP：0x21 0x0b，wValue=0，wIndex=interface，wLength=0；失败才放弃 probe
SET_IDLE	SETUP：0x21 0x0a，同上；失败不致命
Boot Mouse 报告	4 字节：byte0 按钮位图（左/右/中），byte1 X、byte2 Y、byte3 滚轮（均为 signed char）；actual_length>=3 即接受
输入事件	按钮变化 input_report_key(BTN_*)；位移 input_report_rel(REL_X/Y/WHEEL)；末尾 input_sync()

自动装载链. 从设备注册到鼠标驱动绑定，跨越 USB core、设备 core 与模块加载器：

```
usb_device_register(接口1)  ->  device_register()  ->  无驱动匹配
  ->  usb_modalias() = "usb:c03s01p02"
  ->  request_module(alias)                    (见 device.c)
       ->  命中静态 alias 表 ->  queue_work(kmod_wq, slot)
            ->  kmod_run() ->  kmod_load("/lib/modules/usbmouse.ko")
                 ->  begin_op(); namei(); ilock()
                 ->  ip->iop->read(ip, 0, ko_image, 0, sz)   % ext2 分派，非经典 readi()
                 ->  load_module_image(): 模块区分配、符号解析、RELA 重定位
                 ->  run init: usbmouse_driver_init()
                      ->  init_workqueue(usbmouse_wq)        % 日志 "usbmouse_wq worker=0 pid=17"
                      ->  usb_register_driver() -> driver_register()
                           ->  重扫已登记设备 -> usb_match(接口1) 命中
                           ->  usbmouse_probe() -> usb_submit_urb()
                                ->  dwc2_submit_urb() -> dwc2_interrupt_schedule_locked()
                                     若 ch6 空闲则 rx_start，否则排入 interrupt_rx[] 等轮转
```

两个要点。其一，driver_register() 会重新遍历总线上**已存在**的设备，所以“设备先登记并异步排队装载、驱动后注册”的次序仍能绑定接口 1，无需手动 insmod。其二，kmod_load() 里读 .ko 必须走 ip->iop->read（VFS 分派），不能用经典 xv6fs 的 readi()：readi() 的 bmap()->getsb() 假设设备带 xv6 超级块，在 ext2 根上会直接 panic: getsb: device has no xv6 filesystem。本日志的验收点正是这行：

```
request_module: loading /lib/modules/usbmouse.ko
module: loaded usbhid-mouse (1808 bytes)
```

出现这两行、且不再有 getsb panic，说明方案 A 的分接口发布、MODALIAS 自动装载与 ext2 上的模块读取在真机跑通。

本日志尚未证明的部分. 按“只承认证据”的原则，这次链路证明了**自动装载**，却没有证明**鼠标数据端到端接收**：全程未出现 usbmouse: IRQ-driven boot mouse ready，也没有 usbmouse: first report；拔出前是连续四次的 dwc2: HID ch6 error intr=82 (XACTERR | CHHLTD) 与 usbkbd: channel 6 transfer error; rearming。也就是说 full-speed 复合设备在共享 ch6、经 TT 的 split interrupt-IN 调度仍未收敛，键盘与鼠标争用同一 channel 的时序是主要嫌疑；拔出仍触发 connected=a->2 的**整棵拓扑重枚举**，而非只对 port 3 局部增删。下一步诊断应在 usbmouse_probe() 各返回前打印失败的 control 请求，并把 ch6 错误路径的 HCSPLT/HCTSIZ/HCCHAR 一并 dump，以区分是 SET_PROTOCOL 被 STALL、还是 interrupt-IN 的 CSPLIT 窗口错过。